



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	APE 3 – Estructuras de Repetición en Java: resolución de los 10 ejercicios propuestos
Unidad de Organización	Unidad Básica
Curricular:	
Nivel y Paralelo:	Primer semestre – Ingeniería de Software
Alumnos participantes:	Sandra Jazmin Alulema Chato Anthony Daniel Armas Jaya Joselyn Maribel Guaman Sagñay Valeria Valentina Ortiz Mosquera Andrés Gabriel Vayas Masache
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. José Ruben Caiza Caizabuan

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivo General:

Desarrollar soluciones algorítmicas utilizando estructuras repetitivas for, while y do-while, seleccionando el ciclo apropiado según el problema, aplicando validaciones, contadores, acumuladores, centinelas y ciclos anidados.

Específicos:

- Analizar cada problema identificando entradas, procesos y salidas, y representarlo mediante algoritmo, pseudocódigo y diagrama de flujo.
- Implementar en Java los diez ejercicios con estructuras for, while y do-while, incorporando validaciones, contadores, acumuladores y centinelas.
- Comprobar el funcionamiento de cada solución mediante pruebas de escritorio contrastadas con la ejecución del programa.

2.2 Modalidad

Presencial.

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

Desarrollar los 10 ejercicios propuestos en la guía APE 3 utilizando Java. Para cada problema se presenta la secuencia: problema, análisis (entradas, procesos y salidas), algoritmo, pseudocódigo, diagrama de flujo, código Java y prueba de escritorio. El código debe ser legible, estar correctamente implementado y contener comentarios breves en las partes relevantes.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Computador personal.
- Java Development Kit (JDK) para compilar y ejecutar los programas.
- Visual Studio Code como entorno de desarrollo.
- PSeInt para la elaboración del pseudocódigo.
- Repositorio GitHub del equipo (trabajo colaborativo).

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:



- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial

Otros (Especifique): Java (JDK), Visual Studio Code, PSeInt y GitHub

2.6 Actividades por desarrollar

Resolver los siguientes ejercicios de la guía APE 3:

- **Ejercicio 1.** Promedio de calificaciones – estructura: for (con do-while para validar).
- **Ejercicio 2.** Control de edades con centinela – estructura: while.
- **Ejercicio 3.** Calculadora con menú repetitivo – estructura: do-while + switch.
- **Ejercicio 4.** Tabla de multiplicar validada – estructura: while + for.
- **Ejercicio 5.** Cajero universitario – estructura: do-while + switch.
- **Ejercicio 6.** Estadísticas de un curso – estructura: for (con while para validar).
- **Ejercicio 7.** Venta de entradas CineCampus – estructura: do-while (con while para validar).
- **Ejercicio 8.** Estacionamiento universitario – estructura: while.
- **Ejercicio 9.** Matriz lógica de asistencia – estructura: for anidado (con do-while para validar).
- **Ejercicio 10.** Sistema integrado de ventas – estructura: do-while + for/while.

2.7 Resultados obtenidos

A continuación, se presenta la resolución de los diez ejercicios. Cada uno incluye su problema, análisis, algoritmo, pseudocódigo (syntax de PSeInt), diagrama de flujo (simbología ANSI/ISO), código Java y prueba de escritorio. Los resultados de cada prueba de escritorio coinciden con la ejecución real del programa con los mismos datos de entrada.

2.7.1 Ejercicio 1: Promedio de calificaciones

Problema

Desarrollar un programa que solicite la cantidad N de estudiantes y luego registre sus calificaciones, válidas entre 0 y 10. El sistema debe mostrar el promedio general, la calificación mayor, la menor, el número de aprobados y el número de reprobados.

Análisis

Estructura repetitiva utilizada: for (con do-while para validar).

Se usa for porque el número de repeticiones (N) se conoce antes de empezar a registrar notas. La validación de los datos se resuelve con do-while, que pide el dato al menos una vez.

Tabla 1. Entradas, procesos y salidas del Ejercicio 1.

Entradas	Procesos	Salidas
• N (cantidad de estudiantes, entero > 0) • nota de cada estudiante (real entre 0 y 10)	• Validar $N > 0$ y cada nota en $[0, 10]$ • Acumular las notas (suma) • Comparar cada nota para hallar mayor y menor • Contar aprobados ($\text{nota} \geq 7$) y reprobados ($\text{nota} < 7$) • Promedio = suma / N	• Promedio general • Calificación mayor y menor • Número de aprobados y dereprobados

Algoritmo

1. Inicio.
2. Solicitar N; mientras $N \leq 0$ mostrar error y volver a solicitarlo.
3. Inicializar suma = 0, mayor = 0, menor = 10, aprobados = 0, reprobados = 0.



4. Repetir para i desde 1 hasta N:

Solicitar la nota; mientras esté fuera de [0, 10] mostrar error y volver a solicitarla.

suma = suma + nota.

Si nota > mayor, mayor = nota. Si nota < menor, menor = nota.

Si nota ≥ 7, aprobados = aprobados + 1; en caso contrario, reprobados = reprobados + 1.

5. Mostrar suma / N, mayor, menor, aprobados y reprobados.

6. Fin.

Pseudocódigo

```
Algoritmo Ejercicio01
  Definir n, i, aprobados, reprobados Como Entero
  Definir nota, suma, mayor, menor Como Real  suma <- 0; mayor
  <- 0; menor <- 10  aprobados <- 0; reprobados <- 0
  Repetir
    Escribir "Cantidad de estudiantes (N): "
    Leer n
    Si n <= 0 Entonces
      Escribir "Error: N debe ser mayor que 0."
    FinSi
  Hasta Que n > 0
  Para i <- 1 Hasta n Con Paso 1 Hacer
    Repetir
      Escribir "Calificación ", i, ": "
      Leer nota
      Si nota < 0 O nota > 10 Entonces
        Escribir "Error: la nota debe estar entre 0 y 10."
      FinSi
    Hasta Que nota >= 0 Y nota <= 10      suma <- suma +
    nota
    Si nota > mayor Entonces      mayor <- nota
    FinSi
    Si nota < menor Entonces      menor <- nota
    FinSi
    Si nota >= 7 Entonces
      aprobados <- aprobados + 1
    SiNo
      reprobados <- reprobados + 1
    FinSi
  FinPara
  Escribir "Promedio general: ", suma / n
  Escribir "Calificación mayor: ", mayor
  Escribir "Calificación menor: ", menor
  Escribir "Aprobados: ", aprobados
  Escribir "Reprobados: ", reprobados
FinAlgoritmo
```

Diagrama de flujo

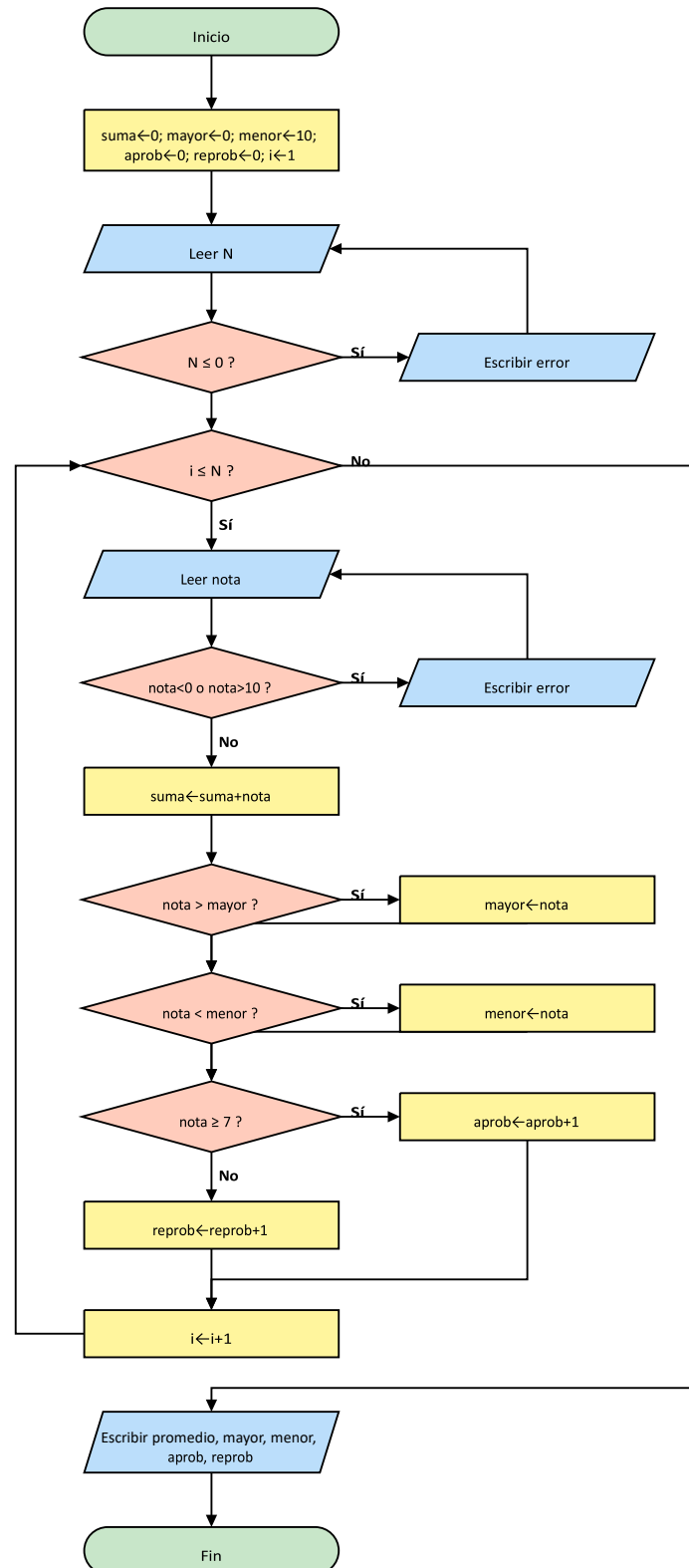


Figura 1. Diagrama de flujo del Ejercicio 1: Promedio de calificaciones.

Código Java

Archivo Ejercicio01.java

```
import java.util.Locale;
import java.util.Scanner;
```



```
public class Ejercicio01 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);
        final double NOTA_APROBACION = 7.0;

        int n;
        do {
            System.out.print("Cantidad de estudiantes (N): ");
            n = sc.nextInt();
            if (n <= 0) {
                System.out.println("Error: N debe ser mayor que 0.");
            }
        } while (n <= 0);

        double suma = 0, mayor = 0, menor = 10;
        int aprobados = 0, reprobados = 0;

        for (int i = 1; i <= n; i++) {
            double nota;
            do {
                System.out.print("Calificacion " + i + ": ");
                nota = sc.nextDouble();
                if (nota < 0 || nota > 10) {
                    System.out.println("Error: la nota debe estar entre 0 y 10.");
                }
            } while (nota < 0 || nota > 10);

            suma += nota;
            if (nota > mayor) mayor = nota;
            if (nota < menor) menor = nota;
            if (nota >= NOTA_APROBACION) aprobados++;
            else reprobados++;
        }

        System.out.printf("Promedio general: %.2f\n", suma / n);
        System.out.printf("Calificacion mayor: %.2f\n", mayor);
        System.out.printf("Calificacion menor: %.2f\n", menor);
        System.out.println("Aprobados: " + aprobados);
        System.out.println("Reprobados: " + reprobados);
    }
}
```

Prueba de escritorio

Datos de prueba: N = 4; notas 8.5, 11 (inválida), 6, 9.5 y 4.

Tabla 2. Prueba de escritorio del Ejercicio 1.

N.º	Entrada	Proceso	Salida
1	N = 4	4 > 0 es válido. suma=0, mayor=0, menor=10, aprob=0, reprob=0	—
2	nota = 8.5 (i=1)	válida; suma=8.5; mayor=8.5; menor=8.5; 8.5 ≥ 7 → aprob=1	—
3	nota = 11 (i=2)	11 > 10 → inválida; se vuelve a solicitar	Error: la nota debe estar entre 0 y 10.
4	nota = 6 (i=2)	suma=14.5; mayor=8.5; menor=6; 6 < 7 → reprob=1	—
5	nota = 9.5 (i=3)	suma=24; mayor=9.5; menor=6; aprob=2	—
6	nota = 4 (i=4)	suma=28; menor=4; reprob=2	—
7	i = 5	5 > N: termina el for. Promedio = 28 / 4 = 7.00	Promedio 7.00; mayor 9.50; menor 4.00; aprobados 2; reprobados 2



2.7.2 Ejercicio 2: Control de edades con centinela

Problema

Ingresar edades válidas de personas. El ingreso terminará cuando se escriba -1. El programa deberá determinar cuántos son menores de edad, adultos y mayores de 65 años, además del promedio de edades ingresadas.

Análisis

Estructura repetitiva utilizada: while.

Se usa while porque no se sabe cuántas edades se ingresarán: el ciclo continúa mientras la edad sea distinta del centinela -1 (lectura inicial + lectura al final de cada vuelta).

Tabla 3. Entradas, procesos y salidas del Ejercicio 2.

Entradas	Procesos	Salidas
• edad (entero); el valor -1 es el centinela	• Validar que la edad esté entre 0 y 120 • Acumular edades y contar las válidas • Clasificar: menor (< 18), adulto (18 a 65), mayor (> 65) • Promedio = suma / total	• Cantidad de menores de edad, adultos y mayores de 65 años • Promedio de edades

Algoritmo

1. Inicio.
2. Inicializar menores, adultos, mayores, total y suma en 0.
3. Leer edad.
4. Mientras edad $\neq$ -1:
 - Si edad < 0 o edad > 120, mostrar error.
 - En caso contrario: suma = suma + edad; total = total + 1.
 - Si edad < 18, menores++; si no, si edad $\leq$ 65, adultos++; si no, mayores++.
 - Leer la siguiente edad.
5. Si total > 0, mostrar menores, adultos, mayores y suma / total; si no, informar que no hubo datos.
6. Fin.



Pseudocódigo

```
Algoritmo Ejercicio02
    Definir edad, menores, adultos, mayores, total, suma Como Entero    menores <- 0; adultos <- 0;
    mayores <- 0    total <- 0; suma <- 0
    Escribir "Edad (-1 para terminar): "
    Leer edad
    Mientras edad <> -1 Hacer
        Si edad < 0 O edad > 120 Entonces
            Escribir "Error: edad no válida."
        SiNo
            suma <- suma + edad            total <- total +
1
            Si edad < 18 Entonces                menores <-
menores + 1
            SiNo
                Si edad <= 65 Entonces                    adultos <- adultos
+ 1
                SiNo
                    mayores <- mayores + 1
                FinSi
            FinSi
        FinSi
    Escribir "Edad (-1 para terminar): "
    Leer edad
    FinMientras
    Si total > 0 Entonces
        Escribir "Menores de edad: ", menores
        Escribir "Adultos: ", adultos
        Escribir "Mayores de 65 años: ", mayores
        Escribir "Promedio de edades: ", suma / total
    SiNo
        Escribir "No se ingresaron edades válidas."
    FinSi
FinAlgoritmo
```

Diagrama de flujo

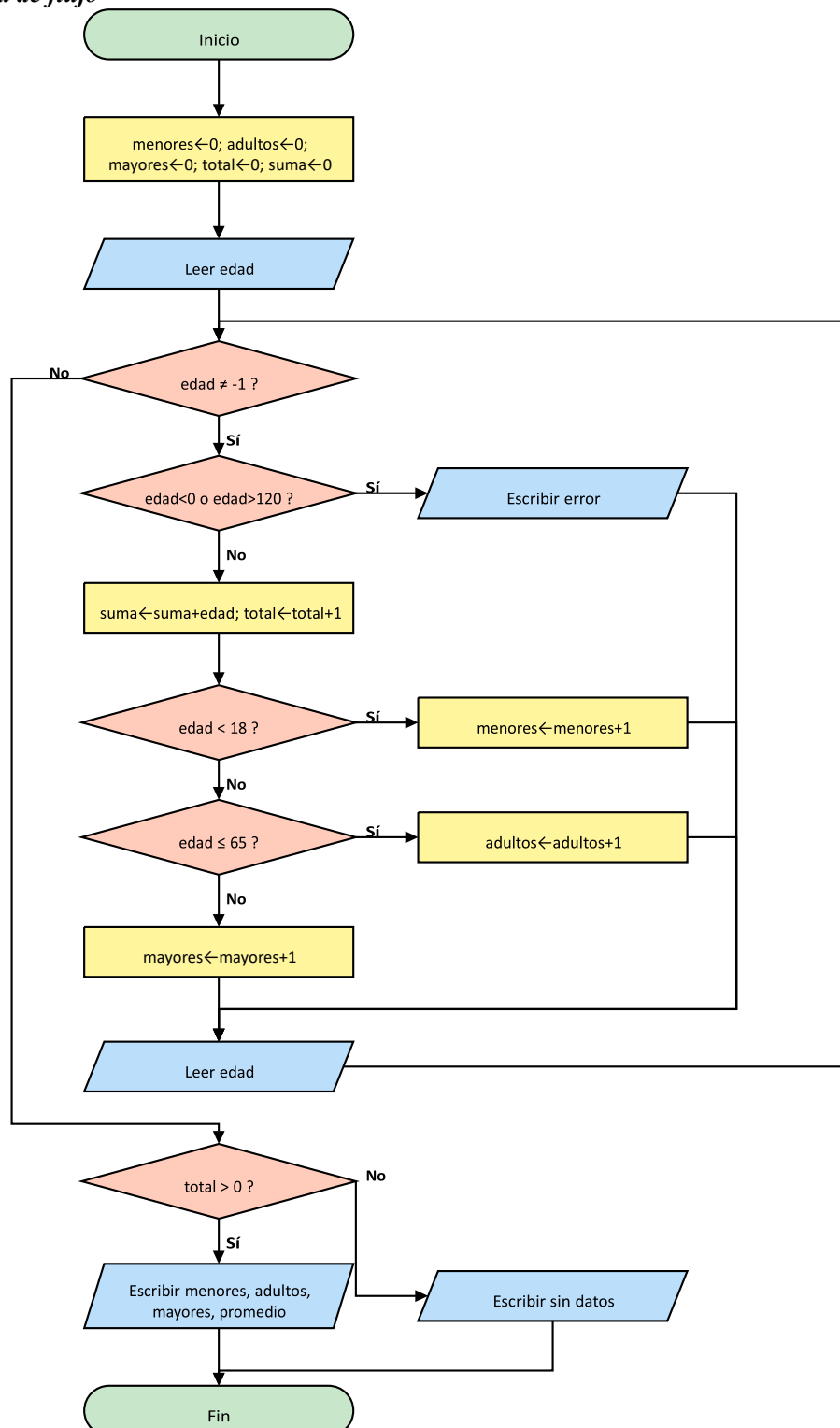


Figura 2. Diagrama de flujo del Ejercicio 2: Control de edades con centinela.

Código Java

Archivo Ejercicio02.java



```
import java.util.Scanner;

public class Ejercicio02 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int menores = 0, adultos = 0, mayores = 0, total = 0, suma = 0;

        System.out.print("Edad (-1 para terminar): ");
        int edad = sc.nextInt();

        while (edad != -1) {
            if (edad < 0 || edad > 120) {
                System.out.println("Error: edad no valida.");
            } else {
                suma += edad;
                total++;

                if (edad < 18) {
                    menores++;
                } else if (edad <= 65) {
                    adultos++;
                } else {
                    mayores++;
                }
            }

            System.out.print("Edad (-1 para terminar): ");
            edad = sc.nextInt();
        }

        if (total > 0) {
            System.out.println("Menores de edad: " + menores);
            System.out.println("Adultos: " + adultos);
            System.out.println("Mayores de 65 años: " + mayores);
            System.out.printf("Promedio de edades: %.2f\n", (double) suma / total);
        } else {
            System.out.println("No se ingresaron edades validas.");
        }
    }
}
```

Prueba de escritorio

Datos de prueba: 15, 30, 70, 130 (inválida), 17, 40 y -1.

Tabla 4. Prueba de escritorio del Ejercicio 2.

N.º	Entrada	Proceso	Salida
1	—	menores=0; adultos=0; mayores=0; total=0; suma=0	—
2	edad = 15	≠ -1; válida; suma=15; total=1; 15 < 18 → menores=1	—



3	edad = 30	suma=45; total=2; $18 \leq 30 \leq 65 \rightarrow$ adultos=1	—
4	edad = 70	suma=115; total=3; $70 > 65 \rightarrow$ mayores=1	—
5	edad = 130	$130 > 120 \rightarrow$ inválida; no se acumula	Error: edad no valida.
6	edad = 17	suma=132; total=4; menores=2	—
7	edad = 40	suma=172; total=5; adultos=2	—
8	edad = -1	centinela: termina el while. Promedio = $172 / 5 = 34.40$	Menores 2; adultos 2; mayores 1; promedio 34.40

2.7.3 Ejercicio 3: Calculadora con menú repetitivo

Problema

Construir un menú con las opciones: 1) Sumar, 2) Restar, 3) Multiplicar, 4) Dividir y 5) Salir. El menú debe repetirse hasta seleccionar Salir. Controlar la división para evitar dividir entre cero.

Análisis

Estructura repetitiva utilizada: do-while + switch.

Se usa do-while porque el menú debe mostrarse al menos una vez y repetirse hasta que el usuario elija Salir; switch selecciona la operación según la opción.

Tabla 5. Entradas, procesos y salidas del Ejercicio 3.

Entradas	Procesos	Salidas
• opción del menú (entero) • primer número a y segundonúmero b (reales)	• Mostrar el menú y leer la opción • Seleccionar la operación con switch • Si la operación es dividir y $b = 0$, mostrar error • Repetir mientras opción $\neq 5$	• Resultado de la operación • Mensajes de error (división entre cero, opción inválida) • Mensaje de fin del programa

Algoritmo

1. Inicio.

2. Repetir:

Mostrar el menú y leer opción.

Si opción está entre 1 y 4: leer a y b y, según la opción, calcular y mostrar el resultado.

Opción 4: si $b = 0$ mostrar error; si no, mostrar a / b .

Si no, si opción $\neq 5$, mostrar «Opción inválida».

3. Mientras opción $\neq 5$.

4. Mostrar «Fin del programa».

5. Fin.



Pseudocódigo

```
Algoritmo Ejercicio03
  Definir opcion Como Entero
  Definir a, b Como Real
  Repetir
    Escribir "1) Sumar 2) Restar 3) Multiplicar 4) Dividir 5) Salir"
    Leer opcion
    Si opcion >= 1 Y opcion <= 4 Entonces
      Leer a, b
      Segun opcion Hacer
        1: Escribir "Resultado: ", a + b
        2: Escribir "Resultado: ", a - b
        3: Escribir "Resultado: ", a * b
        4: Escribir "Resultado: ", a / b
      Si b = 0 Entonces
        Escribir "Error: no se puede dividir entre cero."
      SiNo
        Escribir "Resultado: ", a / b
      FinSi
    FinSegun
  SiNo
    Si opcion <> 5 Entonces
      Escribir "Opción inválida."
    FinSi
  FinSi
Hasta Que opcion = 5
Escribir "Fin del programa."
FinAlgoritmo
```

Diagrama de flujo

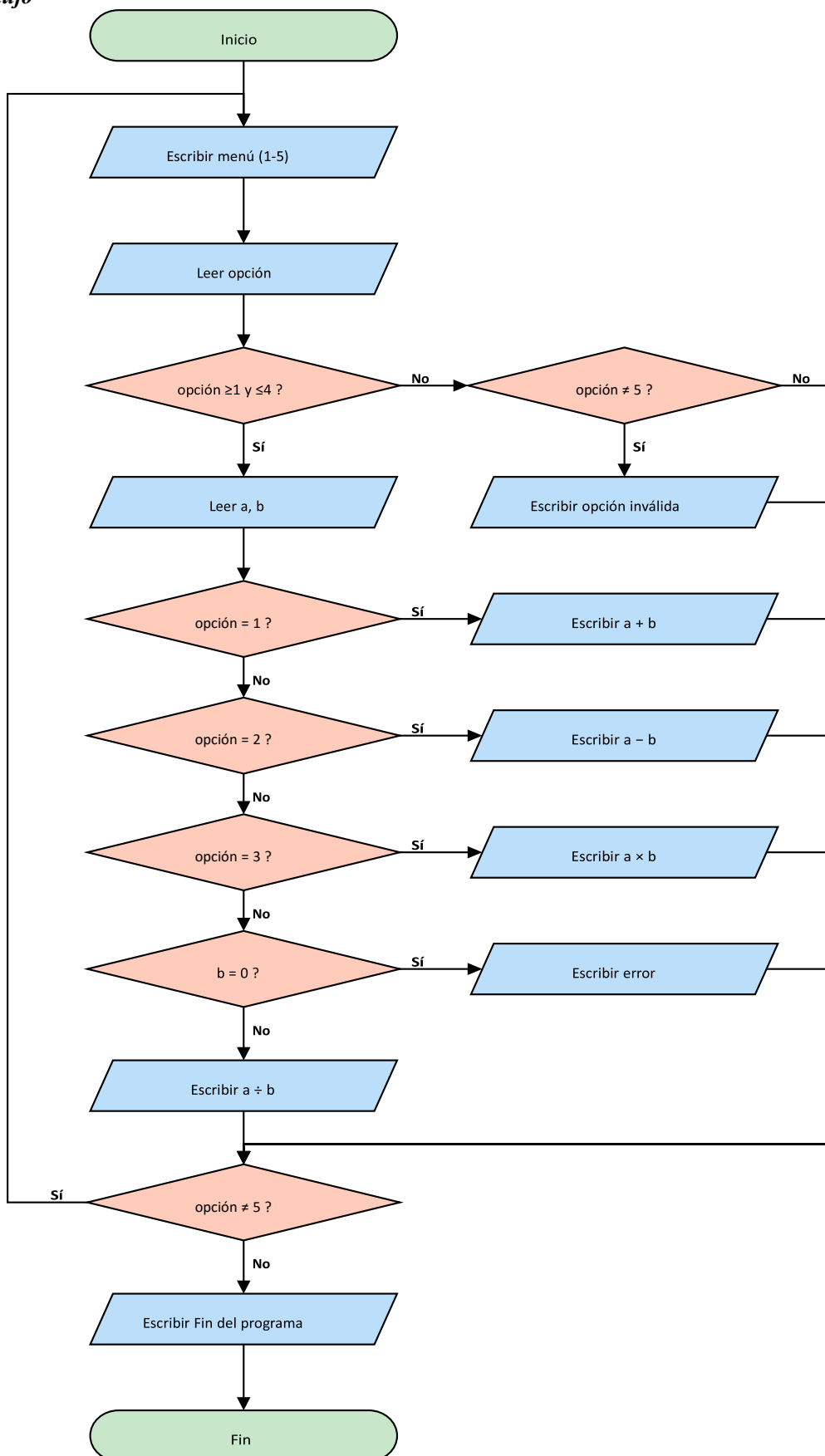


Figura 3. Diagrama de flujo del Ejercicio 3: Calculadora con menú repetitivo.



Código Java

Archivo Ejercicio03.java

```
import java.util.Locale;
import java.util.Scanner;

public class Ejercicio03 {
    public static void main(String[] args) {
        Scanner sc = new
Scanner(System.in).useLocale(Locale.US);
        int opcion;

        do {
            System.out.println("\n---
CALCULADORA ---");
            System.out.println("1) Sumar 2)
Restar 3) Multiplicar 4) Dividir 5) Salir");
            System.out.print("Opcion: ");
            opcion = sc.nextInt();

            if (opcion >= 1 && opcion <= 4) {
                System.out.print("Primer numero:
");
                double a = sc.nextDouble();
                System.out.print("Segundo
numero: ");
                double b = sc.nextDouble();

                switch (opcion) {
                    case 1:

                        System.out.printf("Resultado: %.2f%n", a +
b);
                            break;
                    case 2:

                        System.out.printf("Resultado: %.2f%n", a -
b);
                            break;
                    case 3:

                        System.out.printf("Resultado: %.2f%n", a *
b);
                            break;
                    case 4:
                        if (b == 0) {
                            System.out.println("Error:
no se puede dividir entre cero.");
                        } else {
```



```

System.out.printf("Resultado: %.2f%n", a /
b);
        }
        break;
    }
    } else if (opcion != 5) {
        System.out.println("Opcion
invalida.");
    }
    } while (opcion != 5);

    System.out.println("Fin del
programa.");
}
}

```

Prueba de escritorio

Datos de prueba: (1, 8, 2), (4, 5, 0), (4, 9, 3), 7 y 5.

Tabla 6. Prueba de escritorio del Ejercicio 3.

N.º	Entrada	Proceso	Salida
1	opción = 1; a = 8; b = 2	1 está en [1, 4] → caso 1: $8 + 2 = 10$	Resultado: 10.00
2	opción = 4; a = 5; b = 0	caso 4: $b = 0$ → no se divide	Error: no se puede dividir entre cero.
3	opción = 4; a = 9; b = 3	$b \neq 0 \rightarrow 9 / 3 = 3$	Resultado: 3.00
4	opción = 7	fuera de [1, 4] y distinta de 5	Opcion invalida.
5	opción = 5	opción = 5 → la condición del do-while es falsa; termina	Fin del programa.

2.7.4 Ejercicio 4: Tabla de multiplicar validada

Problema

Solicitar un número entre 1 y 12. Si el dato es incorrecto, deberá volver a solicitarlo. Una vez validado, generar su tabla de multiplicar desde 1 hasta 12.

Análisis

Estructura repetitiva utilizada: while + for.

Se usa while porque no se sabe cuántas veces el usuario se equivocará al ingresar el dato; luego for porque la tabla siempre tiene exactamente 12 repeticiones.

Tabla 7. Entradas, procesos y salidas del Ejercicio 4.

Entradas	Procesos	Salidas
• número (entero entre 1 y 12)	• Validar el rango con while (validación previa) • Recorrer i de 1 a 12 con for • Calcular número $\times$ i	• Mensaje de error si el dato es incorrecto • Las 12 líneas de la tabla demultiplicar

Algoritmo

1. Inicio.
2. Leer número.
3. Mientras número < 1 o número > 12 : mostrar error y volver a leer número.

4. Mostrar «Tabla del número».
5. Repetir para i desde 1 hasta 12: mostrar $\text{número} \times i = \text{resultado}$.
6. Fin.

Pseudocódigo

```

Algoritmo Ejercicio04
    Definir numero, i Como Entero
    Escribir "Número (1 a 12): "
    Leer numero
    Mientras numero < 1 O numero > 12 Hacer
        Escribir "Error: el número debe estar entre 1 y 12."
        Escribir "Número (1 a 12): "
        Leer numero
    FinMientras
    Escribir "Tabla del ", numero
    Para i <- 1 Hasta 12 Con Paso 1 Hacer
        Escribir numero, " x ", i, " = ", numero * i
    FinPara
FinAlgoritmo
    
```

Diagrama de flujo

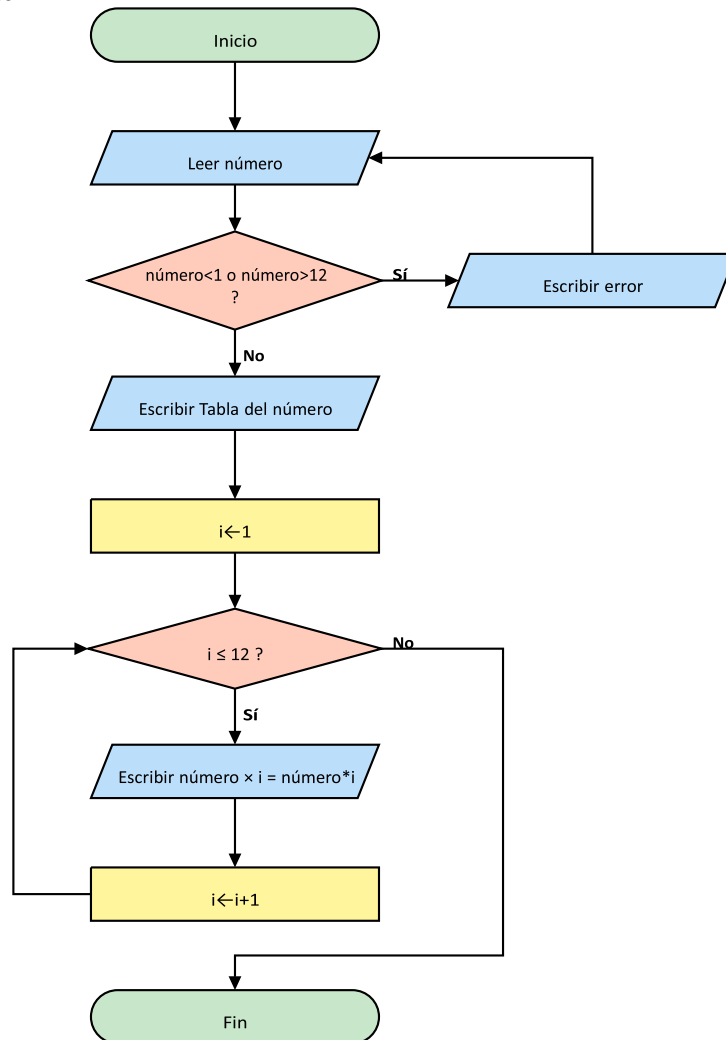


Figura 4. Diagrama de flujo del Ejercicio 4: Tabla de multiplicar validada.

Código Java

Archivo Ejercicio04.java



```
import java.util.Scanner;

public class Ejercicio04 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Numero (1 a 12): ");
        int numero = sc.nextInt();

        while (numero < 1 || numero > 12) {
            System.out.println("Error: el numero debe estar entre 1 y 12.");
            System.out.print("Numero (1 a 12): ");
            numero = sc.nextInt();
        }

        System.out.println("Tabla del " + numero);

        for (int i = 1; i <= 12; i++) {
            System.out.println(numero + " x " + i + " = " + (numero * i));
        }
    }
}
```

Prueba de escritorio

Datos de prueba: 15 (inválido), 0 (inválido) y 7.

Tabla 8. Prueba de escritorio del Ejercicio 4.

N.º	Entrada	Proceso	Salida
1	número = 15	$15 > 12 \rightarrow$ inválido; se vuelve a solicitar	Error: el numero debe estar entre 1 y 12.
2	número = 0	$0 < 1 \rightarrow$ inválido; se vuelve a solicitar	Error: el numero debe estar entre 1 y 12.
3	número = 7	válido; termina el while y se inicia el for	Tabla del 7
4	—	$i = 1: 7 \times 1 = 7$	$7 \times 1 = 7$
5	—	$i = 2: 7 \times 2 = 14$	$7 \times 2 = 14$
6	—	$i = 3: 7 \times 3 = 21$	$7 \times 3 = 21$
7	—	$i = 4: 7 \times 4 = 28$	$7 \times 4 = 28$
8	—	$i = 5: 7 \times 5 = 35$	$7 \times 5 = 35$
9	—	$i = 6: 7 \times 6 = 42$	$7 \times 6 = 42$
10	—	$i = 7: 7 \times 7 = 49$	$7 \times 7 = 49$
11	—	$i = 8: 7 \times 8 = 56$	$7 \times 8 = 56$
12	—	$i = 9: 7 \times 9 = 63$	$7 \times 9 = 63$
13	—	$i = 10: 7 \times 10 = 70$	$7 \times 10 = 70$
14	—	$i = 11: 7 \times 11 = 77$	$7 \times 11 = 77$



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL CARRERA
DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



15	—	$i = 12: 7 \times 12 = 84$	$7 \times 12 = 84$
16	—	$i = 13 > 12$: termina el for	—



2.7.5 Ejercicio 5: Cajero universitario

Problema

Implementar un sistema con saldo inicial y las opciones: consultar saldo, depositar, retirar, ver número de transacciones y salir. No permitir valores negativos ni retiros superiores al saldo disponible.

Análisis

Estructura repetitiva utilizada: do-while + switch.

Se usa do-while porque el menú debe mostrarse al menos una vez y repetirse hasta elegir Salir; el saldo actúa como acumulador y transacciones como contador.

Tabla 9. Entradas, procesos y salidas del Ejercicio 5.

Entradas	Procesos	Salidas
• opción del menú (entero) • monto a depositar o retirar (real)	• Saldo inicial = 100.00; transacciones = 0 • Validar monto > 0 en depósitos y retiros • Validar que el retiro no supere el saldo • Actualizar saldo y contar transacciones exitosas	• Saldo actual • Mensajes de operación exitosa o de error • Número de transacciones • Mensaje de despedida

Algoritmo

1. Inicio.
2. Establecer saldo = 100 y transacciones = 0.
3. Repetir:
 - Mostrar el menú y leer opción.
 - Opción 1: mostrar el saldo.
 - Opción 2: leer depósito; si ≤ 0 mostrar error; si no, saldo = saldo + depósito y transacciones++.
 - Opción 3: leer retiro; si ≤ 0 error; si retiro > saldo «saldo insuficiente»; si no, saldo = saldo – retiro y transacciones++.
 - Opción 4: mostrar transacciones. Opción 5: no hace nada. Otra opción: «Opción inválida».
4. Mientras opción $\neq$ 5.
5. Mostrar el mensaje de despedida.
6. Fin.



Pseudocódigo

```
Algoritmo Ejercicio05
  Definir opcion, transacciones Como Entero   Definir saldo,
  deposito, retiro Como Real   saldo <- 100   transacciones <- 0
  Repetir
    Escribir "1) Consultar saldo 2) Depositar 3) Retirar"
    Escribir "4) Ver transacciones 5) Salir"
    Leer opcion
    Segun opcion Hacer      1:
      Escribir "Saldo actual: $", saldo      2:
      Leer deposito
      Si deposito <= 0 Entonces
        Escribir "Error: el monto debe ser positivo."
      SiNo
        saldo <- saldo + deposito
        transacciones <- transacciones + 1      FinSi      3:
      Leer retiro
      Si retiro <= 0 Entonces
        Escribir "Error: el monto debe ser positivo."
      SiNo
        Si retiro > saldo Entonces
          Escribir "Error: saldo insuficiente."
        SiNo
          saldo <- saldo - retiro      transacciones <- transacciones + 1
        FinSi
      FinSi      4:
      Escribir "Transacciones realizadas: ", transacciones      5:
      // salir: el ciclo termina      De Otro Modo:
      Escribir "Opción inválida."
    FinSegun
  Hasta Que opcion = 5
  Escribir "Gracias por usar el cajero."
FinAlgoritmo
```

Diagrama de flujo

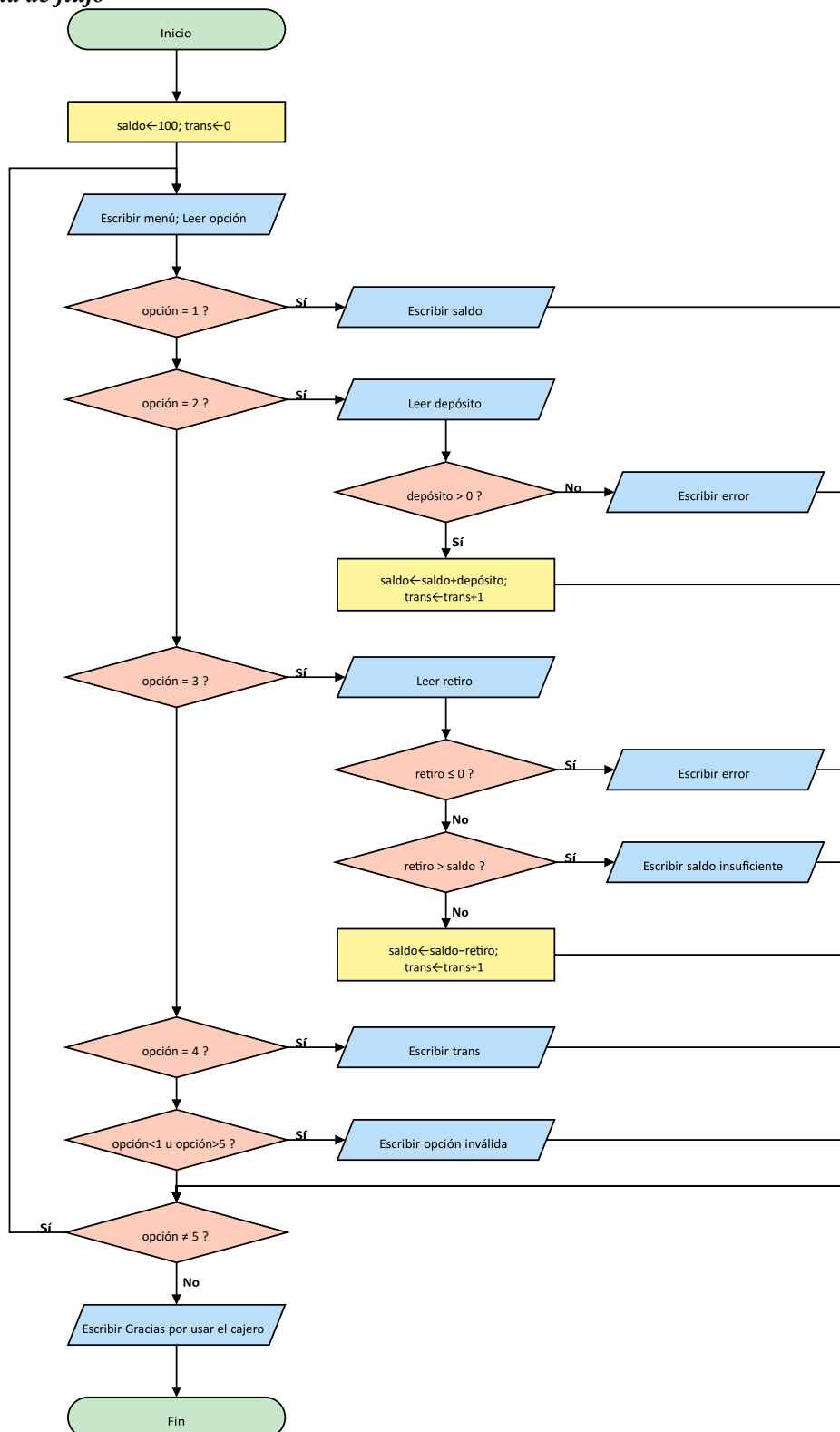


Figura 5. Diagrama de flujo del Ejercicio 5: Cajero universitario.

Código Java

Archivo Ejercicio05.java



```
import java.util.Locale;
import java.util.Scanner;

public class Ejercicio05 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);
        double saldo = 100.00;
        int transacciones = 0;
        int opcion;

        do {
            System.out.println("\n--- CAJERO UNIVERSITARIO ---");
            System.out.println("1) Consultar saldo 2) Depositar 3) Retirar");
            System.out.println("4) Ver transacciones 5) Salir");
            System.out.print("Opcion: ");
            opcion = sc.nextInt();

            switch (opcion) {
                case 1:
                    System.out.printf("Saldo actual: $%.2f\n", saldo);
                    break;

                case 2:
                    System.out.print("Monto a depositar: ");
                    double deposito = sc.nextDouble();

                    if (deposito <= 0) {
                        System.out.println("Error: el monto debe ser positivo.");
                    } else {
                        saldo += deposito;
                        transacciones++;
                        System.out.printf("Deposito exitoso. Saldo: $%.2f\n", saldo);
                    }
                    break;

                case 3:
                    System.out.print("Monto a retirar: ");
                    double retiro = sc.nextDouble();

                    if (retiro <= 0) {
                        System.out.println("Error: el monto debe ser positivo.");
                    } else if (retiro > saldo) {
                        System.out.println("Error: saldo insuficiente.");
                    } else {
                        saldo -= retiro;
                        transacciones++;
                        System.out.printf("Retiro exitoso. Saldo: $%.2f\n", saldo);
                    }
                    break;
            }
        } while (opcion != 5);
    }
}
```



```
case 4:
    System.out.println("Transacciones realizadas: " + transacciones);
    break;

case 5:
    break;

default:
    System.out.println("Opcion invalida.");
}
} while (opcion != 5);

System.out.println("Gracias por usar el cajero.");
}
```

Prueba de escritorio

Datos de prueba: saldo inicial 100.00; opciones 1, 2 (50), 3 (200), 3 (30), 2 (-5), 4, 1 y 5.

Tabla 10. Prueba de escritorio del Ejercicio 5.

N.º	Entrada	Proceso	Salida
1	opción = 1	muestra el saldo (100.00)	Saldo actual: \$100.00
2	opción = 2; monto = 50	$50 > 0 \rightarrow \text{saldo} = 100 + 50 = 150$; trans = 1	Deposito exitoso. Saldo: \$150.00
3	opción = 3; monto = 200	$200 > 0$ pero $200 > 150 \rightarrow$ se rechaza	Error: saldo insuficiente.
4	opción = 3; monto = 30	$30 \leq 150 \rightarrow \text{saldo} = 150 - 30 = 120$; trans = 2	Retiro exitoso. Saldo: \$120.00
5	opción = 2; monto = -5	$-5 \leq 0 \rightarrow$ se rechaza; no cuenta como transacción	Error: el monto debe ser positivo.
6	opción = 4	muestra el contador (2)	Transacciones realizadas: 2
7	opción = 1	muestra el saldo (120.00)	Saldo actual: \$120.00
8	opción = 5	opción = 5 $\rightarrow$ condición falsa; termina el do-while	Gracias por usar el cajero.



2.7.6 Ejercicio 6: Estadísticas de un curso

Problema

Registrar las calificaciones de N estudiantes. Calcular promedio general, nota mayor, nota menor, cantidad y porcentaje de aprobados y reprobados. Validar que todas las notas estén entre 0 y 10.

Análisis

Estructura repetitiva utilizada: for (con while para validar).

Se usa for porque se conoce N; las validaciones con while repiten la lectura mientras el dato sea incorrecto. Se agregan porcentajes respecto al Ejercicio 1.

Tabla 11. Entradas, procesos y salidas del Ejercicio 6.

Entradas	Procesos	Salidas
• N (cantidad de estudiantes, entero > 0) • nota de cada estudiante (real entre 0 y 10)	• Validar $N > 0$ y cada nota en $[0, 10]$ con while • Acumular notas; hallar mayor y menor • Contar aprobados (≥ 7) y reprobados (< 7) • Porcentaje = cantidad $\times 100 / N$	• Promedio general, nota mayor y nota menor • Cantidad y porcentaje de aprobados • Cantidad y porcentaje de reprobados

Algoritmo

1. Inicio.
2. Leer N; mientras $N \leq 0$ mostrar error y volver a leer.
3. Inicializar suma = 0, mayor = 0, menor = 10, aprobados = 0, reprobados = 0.
4. Repetir para i desde 1 hasta N:
 - Leer nota; mientras nota < 0 o nota > 10 mostrar error y volver a leer.
 - suma = suma + nota; actualizar mayor y menor.
 - Si nota ≥ 7 , aprobados++; si no, reprobados++.
5. Calcular promedio = suma / N; %aprobados = aprobados $\times 100 / N$; %reprobados = reprobados $\times 100 / N$.
6. Mostrar todos los resultados.
7. Fin.



Pseudocódigo

```
Algoritmo Ejercicio06
  Definir n, i, aprobados, reprobados Como Entero
  Definir nota, suma, mayor, menor Como Real
  Escribir "Número de estudiantes: "
  Leer n
  Mientras n <= 0 Hacer
    Escribir "Error: debe ser mayor que 0."
    Escribir "Número de estudiantes: "
    Leer n
  FinMientras
  suma <- 0; mayor <- 0; menor <- 10   aprobados <- 0; reprobados
  <- 0
  Para i <- 1 Hasta n Con Paso 1 Hacer
    Escribir "Nota del estudiante ", i, ": "
    Leer nota
    Mientras nota < 0 O nota > 10 Hacer
      Escribir "Error: la nota debe estar entre 0 y 10."
      Escribir "Nota del estudiante ", i, ": "
      Leer nota
    FinMientras
    suma <- suma + nota
    Si nota > mayor Entonces          mayor <- nota
    FinSi
    Si nota < menor Entonces          menor <- nota
    FinSi
    Si nota >= 7 Entonces
      aprobados <- aprobados + 1
    SiNo
      reprobados <- reprobados + 1
    FinSi
  FinPara
  Escribir "Promedio general: ", suma / n
  Escribir "Nota mayor: ", mayor
  Escribir "Nota menor: ", menor
  Escribir "Aprobados: ", aprobados, " (", aprobados * 100 / n, "%)"   Escribir "Reprobados: ", reprobados, " (",
reprobados * 100 / n, "%)"
FinAlgoritmo
```

Diagrama de flujo

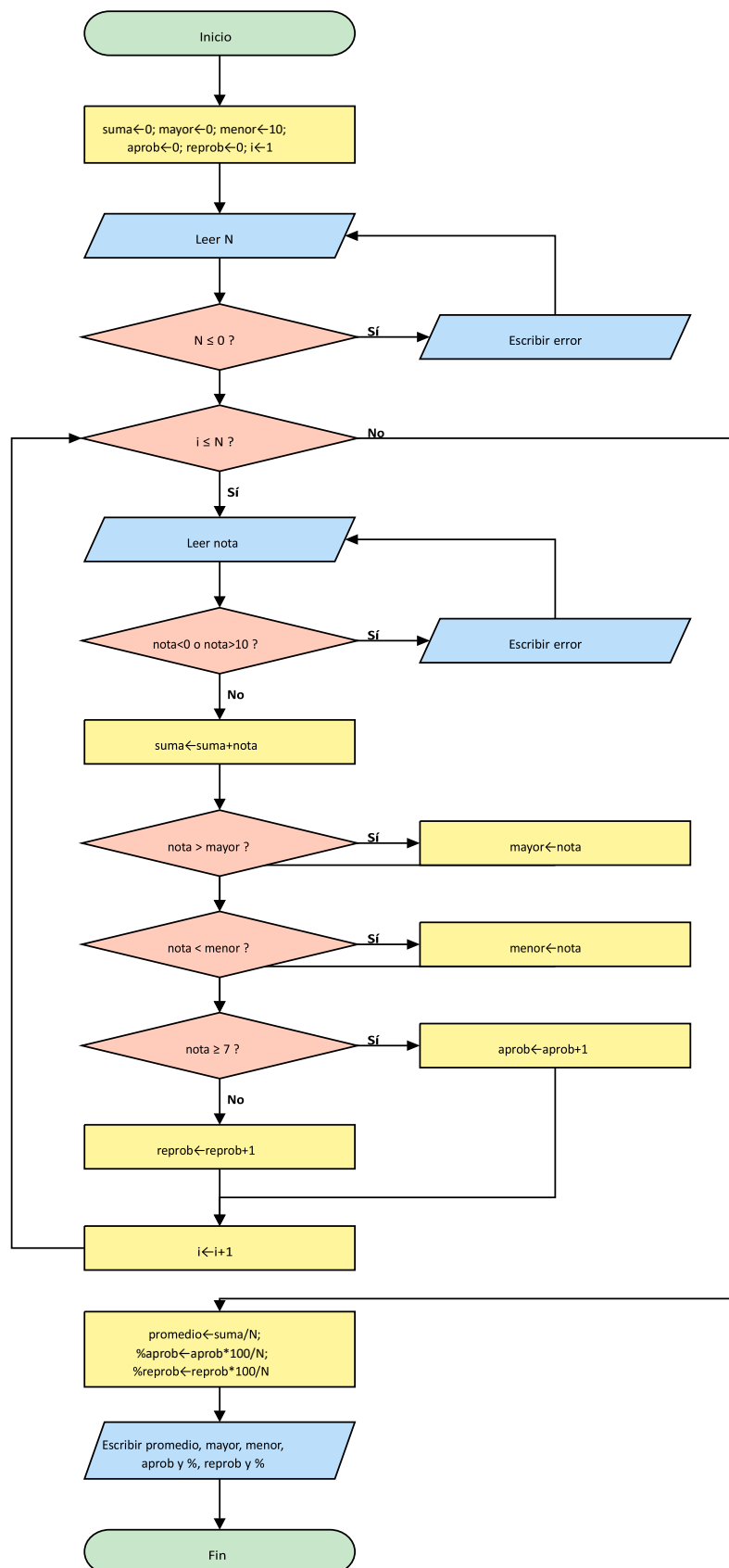


Figura 6. Diagrama de flujo del Ejercicio 6: Estadísticas de un curso.



Código Java

Archivo Ejercicio06.java

```
import java.util.Locale;
import java.util.Scanner;

public class Ejercicio06 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);
        final double NOTA_APROBACION = 7.0;

        System.out.print("Numero de estudiantes: ");
        int n = sc.nextInt();

        while (n <= 0) {
            System.out.println("Error: debe ser mayor que 0.");
            System.out.print("Numero de estudiantes: ");
            n = sc.nextInt();
        }

        double suma = 0, mayor = 0, menor = 10;
        int aprobados = 0, reprobados = 0;

        for (int i = 1; i <= n; i++) {
            System.out.print("Nota del estudiante " + i + ": ");
            double nota = sc.nextDouble();

            while (nota < 0 || nota > 10) {
                System.out.println("Error: la nota debe estar entre 0 y 10.");
                System.out.print("Nota del estudiante " + i + ": ");
                nota = sc.nextDouble();
            }

            suma += nota;
            if (nota > mayor) mayor = nota;
            if (nota < menor) menor = nota;

            if (nota >= NOTA_APROBACION) aprobados++;
            else reprobados++;
        }

        System.out.printf("Promedio general: %.2f\n", suma / n);
        System.out.printf("Nota mayor: %.2f\n", mayor);
        System.out.printf("Nota menor: %.2f\n", menor);
        System.out.printf("Aprobados: %d (%.2f%%)\n", aprobados, aprobados * 100.0 / n);
        System.out.printf("Reprobados: %d (%.2f%%)\n", reprobados, reprobados * 100.0 /
n);
    }
}
```

Prueba de escritorio

Datos de prueba: N = 5; notas 9, 5.5, 7, 11 (inválida), 10 y 3.



Tabla 12. Prueba de escritorio del Ejercicio 6.

N.º	Entrada	Proceso	Salida
1	N = 5	$5 > 0$ válido. suma=0, mayor=0, menor=10, aprob=0, reprob=0	—
2	nota = 9 (i=1)	suma=9; mayor=9; menor=9; $9 \geq 7 \rightarrow$ aprob=1	—
3	nota = 5.5 (i=2)	suma=14.5; menor=5.5; $5.5 < 7 \rightarrow$ reprob=1	—
4	nota = 7 (i=3)	suma=21.5; $7 \geq 7 \rightarrow$ aprob=2	—
5	nota = 11 (i=4)	$11 > 10 \rightarrow$ inválida; se vuelve a solicitar	Error: la nota debe estar entre 0 y 10.
6	nota = 10 (i=4)	suma=31.5; mayor=10; aprob=3	—
7	nota = 3 (i=5)	suma=34.5; menor=3; reprob=2	—
8	i = 6	$6 > N$: termina el for. Promedio = $34.5 / 5 = 6.90$; $3 \times 100/5 = 60\%$; $2 \times 100/5 = 40\%$	Promedio 6.90; mayor 10.00; menor 3.00; aprobados 3 (60.00%); reprobados 2 (40.00%)

2.7.7 Ejercicio 7: Venta de entradas CineCampus

Problema

Registrar de manera repetitiva ventas de entradas. Para cada venta solicitar tipo de entrada, cantidad y precio. Calcular subtotal por venta y total acumulado. Después de cada registro preguntar si desea realizar otra venta.

Análisis

Estructura repetitiva utilizada: do-while (con while para validar).

Se usa do-while porque siempre debe registrarse al menos una venta y luego se pregunta si continúa; el total es un acumulador y nVentas un contador.

Tabla 13. Entradas, procesos y salidas del Ejercicio 7.

Entradas	Procesos	Salidas
• tipo de entrada (1 General, 2 Estudiante, 3 Niño) • cantidad (entero > 0) • precio unitario (real > 0) • respuesta S/N para otra venta	• Validar tipo, cantidad y precio con while • subtotal = cantidad $\times$ precio • total = total + subtotal; nVentas = nVentas + 1 • Repetir mientras la respuesta sea S	• Subtotal de cada venta • Número de ventas realizadas • Total acumulado

Algoritmo

- Inicio.
- Establecer total = 0 y nVentas = 0.
- Repetir:
 - Leer tipo (mientras no sea 1, 2 o 3 mostrar error y volver a leer).
 - Leer cantidad (mientras sea ≤ 0 mostrar error y volver a leer).
 - Leer precio (mientras sea ≤ 0 mostrar error y volver a leer).
 - subtotal = cantidad $\times$ precio; total = total + subtotal; nVentas = nVentas + 1.
 - Mostrar el subtotal y preguntar si desea otra venta (S/N).
- Mientras la respuesta sea S (mayúscula o minúscula).
- Mostrar nVentas y total.
- Fin.



Pseudocódigo

```
Algoritmo Ejercicio07
  Definir tipo, cantidad, nVentas Como Entero
  Definir precio, subtotal, total Como Real
  Definir otra Como Caracter  total <- 0
nVentas <- 0
Repetir
  Escribir "Tipo: 1) General 2) Estudiante 3) Niño"
  Leer tipo
  Mientras tipo < 1 O tipo > 3 Hacer
    Escribir "Tipo inválido. Ingrese 1, 2 o 3: "
    Leer tipo
  FinMientras
  Escribir "Cantidad de entradas: "
  Leer cantidad
  Mientras cantidad <= 0 Hacer
    Escribir "Cantidad inválida. Ingrese un valor mayor que 0: "
    Leer cantidad
  FinMientras
  Escribir "Precio unitario: $"
  Leer precio
  Mientras precio <= 0 Hacer
    Escribir "Precio inválido. Ingrese un valor mayor que 0: $"
    Leer precio
  FinMientras
  subtotal <- cantidad * precio  total <- total +
  subtotal  nVentas <- nVentas + 1
  Escribir "Subtotal de la venta: $", subtotal
  Escribir "¿Desea realizar otra venta? (S/N): "
  Leer otra
  Hasta Que Mayusculas(otra) <> "S"
  Escribir "Ventas realizadas: ", nVentas
  Escribir "Total acumulado: $", total
FinAlgoritmo
```

Diagrama de flujo

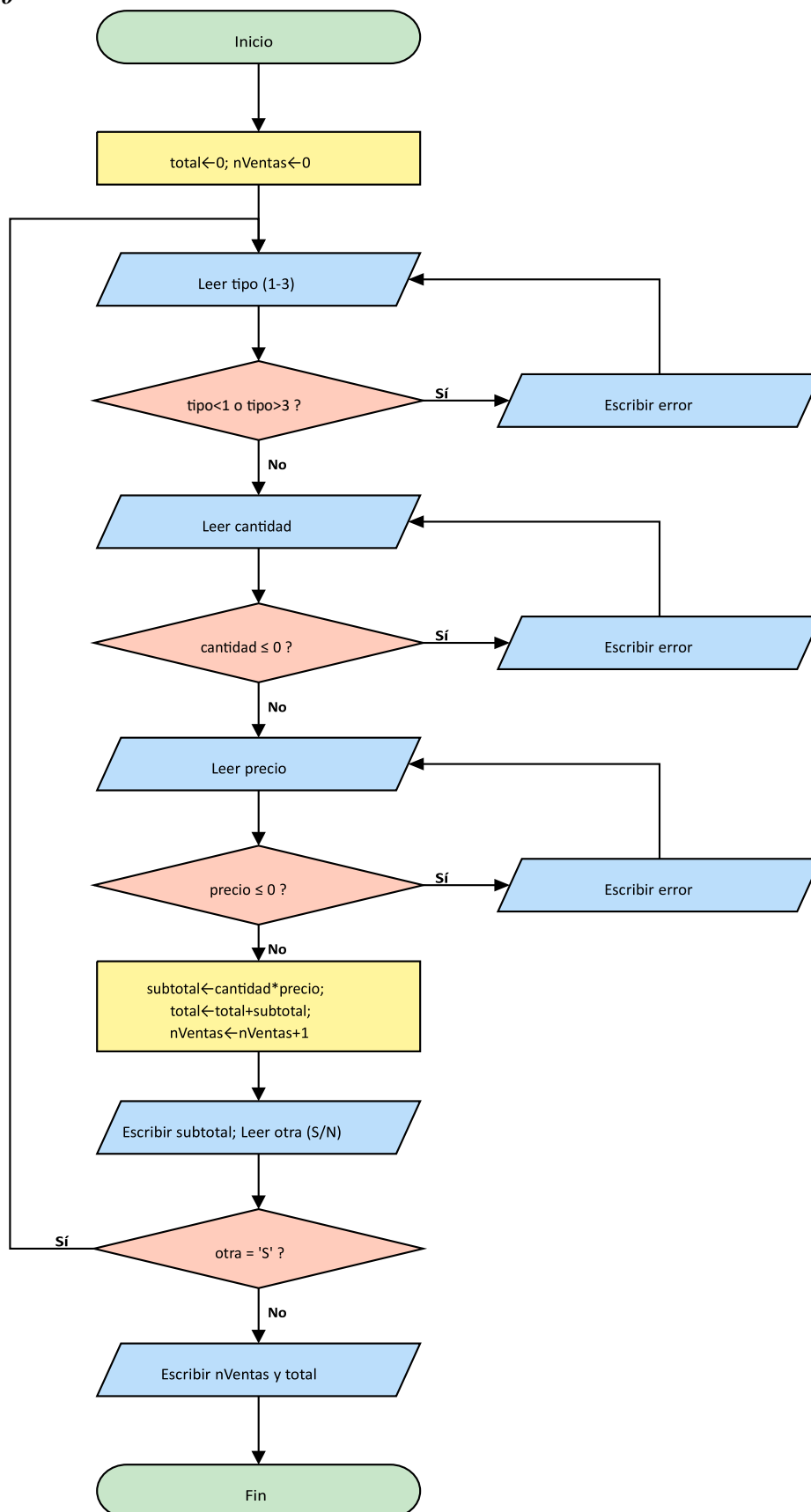


Figura 7. Diagrama de flujo del Ejercicio 7: Venta de entradas CineCampus.



Código Java

Archivo Ejercicio07.java

```
import java.util.Locale;
import java.util.Scanner;

public class Ejercicio07 {
    public static void main(String[] args) {
        Scanner sc = new
Scanner(System.in).useLocale(Locale.US);
        double totalAcumulado = 0;
        int numeroVentas = 0;
        char otra;

        do {
            System.out.println("\nTipo de
entrada: 1) General 2) Estudiante 3) Nino");
            System.out.print("Tipo: ");
            int tipo = sc.nextInt();

            while (tipo < 1 || tipo > 3) {
                System.out.print("Tipo invalido.
Ingrese 1, 2 o 3: ");
                tipo = sc.nextInt();
            }

            System.out.print("Cantidad de
entradas: ");
            int cantidad = sc.nextInt();

            while (cantidad <= 0) {
                System.out.print("Cantidad
invalida. Ingrese un valor mayor que 0: ");
                cantidad = sc.nextInt();
            }

            System.out.print("Precio unitario:
$");
            double precio = sc.nextDouble();

            while (precio <= 0) {
                System.out.print("Precio invalido.
Ingrese un valor mayor que 0: $");
                precio = sc.nextDouble();
            }

            double subtotal = cantidad * precio;
            totalAcumulado += subtotal;
            numeroVentas++;
        }
    }
}
```



```

        System.out.printf("Subtotal de la
venta: $%.2f\n", subtotal);

        System.out.print("Desea realizar otra
venta? (S/N): ");
        otra =
Character.toUpperCase(sc.next().charAt(0));

        } while (otra == 'S');

        System.out.println("\nVentas
realizadas: " + numeroVentas);
        System.out.printf("Total acumulado:
$%.2f\n", totalAcumulado);
    }
}

```

Prueba de escritorio

Datos de prueba: (1, 2, 5.00, s), (2, 3, 3.50, S) y (3, 1, 2.50, N).

Tabla 14. Prueba de escritorio del Ejercicio 7.

N.º	Entrada	Proceso	Salida
1	—	total = 0; nVentas = 0	—
2	tipo = 1; cant = 2; precio = 5.00	datos válidos; subtotal = $2 \times 5.00 = 10.00$; total = 10.00; nVentas = 1	Subtotal de la venta: \$10.00
3	otra = s	Character.toUpperCase → 'S' → se repite el ciclo	—
4	tipo = 2; cant = 3; precio = 3.50	subtotal = $3 \times 3.50 = 10.50$; total = 20.50; nVentas = 2	Subtotal de la venta: \$10.50
5	otra = S	'S' → se repite el ciclo	—
6	tipo = 3; cant = 1; precio = 2.50	subtotal = 2.50; total = 23.00; nVentas = 3	Subtotal de la venta: \$2.50
7	otra = N	'N' → condición falsa; termina el do-while	Ventas realizadas: 3; Total acumulado: \$23.00

2.7.8 Ejercicio 8: Estacionamiento universitario

Problema

Registrar varios vehículos indicando tipo, número de horas y tarifa correspondiente. Calcular el valor individual y la recaudación total. El proceso finalizará al ingresar una opción centinela definida por el equipo.

Análisis

Estructura repetitiva utilizada: while.

Se usa while porque no se conoce cuántos vehículos llegarán: el ciclo continúa mientras el tipo sea distinto del centinela definido por el equipo (0).

Tabla 15. Entradas, procesos y salidas del Ejercicio 8.

Entradas	Procesos	Salidas
----------	----------	---------



• tipo de vehículo (1 Auto, 2 Moto, 3 Camión; 0 = centinela) • horas de estacionamiento (entero > 0)	• Validar tipo y horas • Asignar la tarifa por hora según el tipo (Auto \$1.00, Moto \$0.50, Camión \$2.00) • valor = horas × tarifa • Acumular la recaudación y contar los vehículos	• Valor a pagar por vehículo • Vehículos atendidos • Recaudación total
--	--	--

Algoritmo

- Inicio.
- Establecer recaudación = 0 y vehículos = 0.
- Mostrar el menú y leer tipo.
- Mientras tipo ≠ 0:
 - Si tipo < 1 o tipo > 3, mostrar «Tipo inválido».
 - Si no: leer horas; si horas ≤ 0, mostrar error.
 - Si no: tarifa = 1.00 (Auto), 0.50 (Moto) o 2.00 (Camión).
 - valor = horas × tarifa; recaudación = recaudación + valor; vehículos++; mostrar valor.
 - Mostrar el menú y leer el siguiente tipo.
- Mostrar vehículos atendidos y recaudación total.
- Fin.

Pseudocódigo

```

Algoritmo Ejercicio08
  Definir tipo, horas, vehiculos Como Entero
  Definir tarifa, valor, total Como Real  total <- 0
  vehiculos <- 0
  Escribir "Tipo: 1) Auto 2) Moto 3) Camión 0) Terminar"
  Leer tipo
  Mientras tipo <> 0 Hacer
    Si tipo < 1 O tipo > 3 Entonces
      Escribir "Tipo inválido."
    SiNo
      Escribir "Horas de estacionamiento: "
      Leer horas
      Si horas <= 0 Entonces
        Escribir "Error: las horas deben ser mayores que 0."
      SiNo
        Si tipo = 1 Entonces          tarifa <- 1.00
        SiNo
          Si tipo = 2 Entonces          tarifa <- 0.50
          SiNo
            tarifa <- 2.00
          FinSi
        FinSi
        valor <- horas * tarifa          total <- total + valor
        vehiculos <- vehiculos + 1
        Escribir "Valor a pagar: $", valor
      FinSi
    FinSi
    Escribir "Tipo: 1) Auto 2) Moto 3) Camión 0) Terminar"
    Leer tipo
  FinMientras
  Escribir "Vehiculos atendidos: ", vehiculos
  Escribir "Recaudación total: $", total
FinAlgoritmo
  
```

Diagrama de flujo

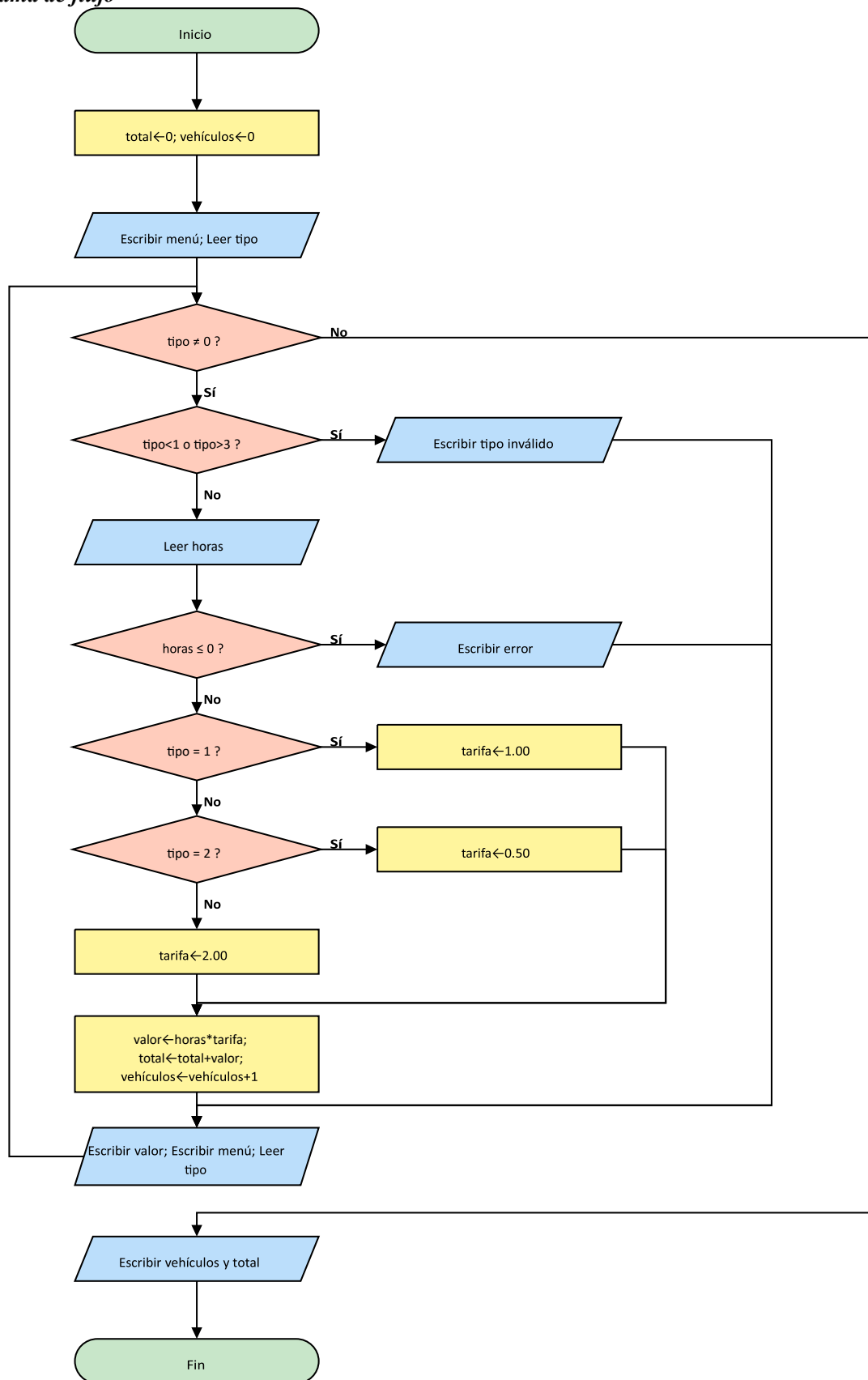


Figura 8. Diagrama de flujo del Ejercicio 8: Estacionamiento universitario.



Código Java

Archivo Ejercicio08.java

```
import java.util.Scanner;

public class Ejercicio08 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        final double TARIFA_AUTO = 1.00;
        final double TARIFA_MOTO = 0.50;
        final double TARIFA_CAMION = 2.00;

        double recaudacion = 0;
        int vehiculos = 0;

        System.out.println("Tipo: 1) Auto 2) Moto 3)
Camion 0) Terminar");
        System.out.print("Tipo: ");
        int tipo = sc.nextInt();

        while (tipo != 0) {
            if (tipo < 1 || tipo > 3) {
                System.out.println("Tipo invalido.");
            } else {
                System.out.print("Horas de
estacionamiento: ");
                int horas = sc.nextInt();

                if (horas <= 0) {
                    System.out.println("Error: las horas
deben ser mayores que 0.");
                } else {
                    double tarifa;

                    if (tipo == 1) {
                        tarifa = TARIFA_AUTO;
                    } else if (tipo == 2) {
                        tarifa = TARIFA_MOTO;
                    } else {
                        tarifa = TARIFA_CAMION;
                    }

                    double valor = horas * tarifa;
                    recaudacion += valor;
                    vehiculos++;

                    System.out.printf("Valor a pagar:
$%.2f\n", valor);
                }
            }
        }
    }
}
```

```

        System.out.println("\nTipo: 1) Auto 2)
Moto 3) Camion 0) Terminar");
        System.out.print("Tipo: ");
        tipo = sc.nextInt();
    }

    System.out.println("Vehiculos atendidos: " +
vehiculos);
    System.out.printf("Recaudacion total:
$%.2f\n", recaudacion);
}
}

```

Prueba de escritorio

Datos de prueba: (1, 3 h), (2, 4 h), (3, 0 h — inválido), (3, 2 h) y 0.

Tabla 16. Prueba de escritorio del Ejercicio 8.

N.º	Entrada	Proceso	Salida
1	—	total = 0; vehículos = 0	Menú de tipos
2	tipo = 1; horas = 3	≠ 0; tipo válido; horas > 0; tarifa = 1.00; valor = 3 × 1.00 = 3.00; total = 3.00; veh = 1	Valor a pagar: \$3.00
3	tipo = 2; horas = 4	tarifa = 0.50; valor = 2.00; total = 5.00; veh = 2	Valor a pagar: \$2.00
4	tipo = 3; horas = 0	horas ≤ 0 → no se registra el vehículo	Error: las horas deben ser mayores que 0.
5	tipo = 3; horas = 2	tarifa = 2.00; valor = 4.00; total = 9.00; veh = 3	Valor a pagar: \$4.00
6	tipo = 0	centinela: termina el while	Vehículos atendidos: 3; Recaudación total: \$9.00

2.7.9 Ejercicio 9: Matriz lógica de asistencia

Problema

Solicitar el número de estudiantes y el número de días. Para cada estudiante registrar su asistencia mediante P (presente) o A (ausente). Al finalizar mostrar las asistencias y ausencias de cada estudiante y los totales del curso.

Análisis

Estructura repetitiva utilizada: for anidado (con do-while para validar).

Se usa for anidado porque se conocen tanto el número de estudiantes como el de días: el for interno se ejecuta completo por cada vuelta del for externo.

Tabla 17. Entradas, procesos y salidas del Ejercicio 9.

Entradas	Procesos	Salidas
- número de estudiantes y número de días (enteros > 0) - marca de asistencia por día: P o A	- Ciclo externo: recorre a cada estudiante - Ciclo interno: recorre los días de ese estudiante - Validar que la marca sea P o A - Contar presentes y ausentes por estudiante y en total	- Asistencias y ausencias de cada estudiante - Totales de asistencias y ausencias del curso

Algoritmo

- Inicio.
- Leer estudiantes y días (ambos deben ser mayores que 0).
- Establecer totalP = 0 y totalA = 0.
- Repetir para e desde 1 hasta estudiantes:



Establecer presentes = 0 y ausentes = 0.

Repetir para d desde 1 hasta días:

Leer marca hasta que sea P o A.

Si marca = P, presentes++; si no, ausentes++.

Mostrar presentes y ausentes del estudiante e; sumarlos a totalP y totalA.

5. Mostrar totalP y totalA.

6. Fin.

Pseudocódigo

```
Algoritmo Ejercicio09
  Definir estudiantes, dias, e, d Como Entero
  Definir presentes, ausentes, totalP, totalA Como Entero
  Definir marca Como Caracter
  Repetir
    Escribir "Número de estudiantes: "
    Leer estudiantes
  Hasta Que estudiantes > 0
  Repetir
    Escribir "Número de días: "
    Leer dias
  Hasta Que dias > 0
  totalP <- 0
  totalA <- 0
  Para e <- 1 Hasta estudiantes Con Paso 1 Hacer
    presentes <- 0
    ausentes <- 0
    Para d <- 1 Hasta dias Con Paso 1 Hacer
      Repetir
        Escribir "Estudiante ", e, ", día ", d, " (P/A): "
        Leer marca
        marca <- Mayusculas(marca)
        Si marca <> "P" Y marca <> "A" Entonces
          Escribir "Error: ingrese solo P o A."
        FinSi
        Hasta Que marca = "P" O marca = "A"
        Si marca = "P" Entonces
          presentes <- presentes + 1
        SiNo
          ausentes <- ausentes + 1
        FinSi
      FinPara
    Escribir "Estudiante ", e, " -> Asistencias: ", presentes, " | Ausencias: ", ausentes
    totalP <- totalP + presentes
    totalA <- totalA + ausentes
  FinPara
  Escribir "TOTAL DEL CURSO -> Asistencias: ", totalP, " | Ausencias: ", totalA
FinAlgoritmo
```

Diagrama de flujo

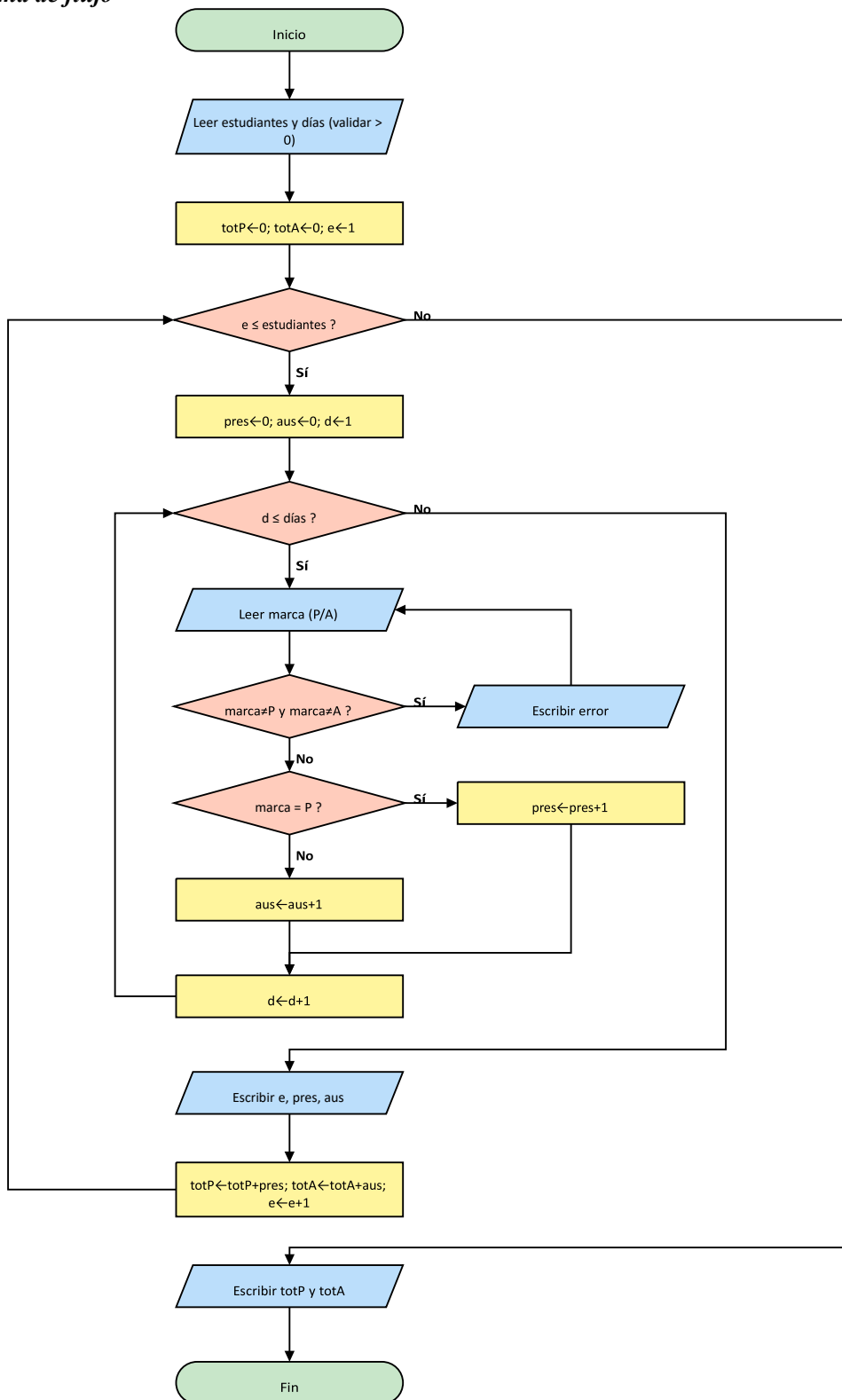


Figura 9. Diagrama de flujo del Ejercicio 9: Matriz lógica de asistencia.

Código Java

Archivo Ejercicio09.java



```
import java.util.Scanner;

public class Ejercicio09 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Numero de estudiantes: ");
        int estudiantes = sc.nextInt();

        while (estudiantes <= 0) {
            System.out.print("Debe ser mayor que 0. Numero de estudiantes: ");
            estudiantes = sc.nextInt();
        }

        System.out.print("Numero de dias: ");
        int dias = sc.nextInt();

        while (dias <= 0) {
            System.out.print("Debe ser mayor que 0. Numero de dias: ");
            dias = sc.nextInt();
        }

        int totalPresentes = 0, totalAusentes = 0;

        for (int e = 1; e <= estudiantes; e++) {
            int presentes = 0, ausentes = 0;

            for (int d = 1; d <= dias; d++) {
                char marca;

                do {
                    System.out.print("Estudiante " + e + ", dia " + d + " (P/A): ");
                    marca = Character.toUpperCase(sc.next().charAt(0));

                    if (marca != 'P' && marca != 'A') {
                        System.out.println("Error: ingrese solo P o A.");
                    }
                } while (marca != 'P' && marca != 'A');

                if (marca == 'P') presentes++;
                else ausentes++;
            }

            System.out.println("Estudiante " + e + " -> Asistencias: " +
                presentes
                + " | Ausencias: " + ausentes);

            totalPresentes += presentes;
            totalAusentes += ausentes;
        }

        System.out.println("TOTAL DEL CURSO -> Asistencias: " +
            totalPresentes
            + " | Ausencias: " + totalAusentes);
    }
}
```

Prueba de escritorio

Datos de prueba: 3 estudiantes y 4 días. E1: P P A P; E2: A A x(inválida) P P; E3: P P P P.

Tabla 18. Prueba de escritorio del Ejercicio 9.

N.º	Entrada	Proceso	Salida
1	estudiantes = 3; días = 4	ambos > 0 → válidos	—
2	—	totP = 0; totA = 0	—
3	—	e = 1: pres = 0; aus = 0; d = 1	—
4	marca = P (e=1, d=1)	presente → pres = 1; aus = 0	—
5	marca = P (e=1, d=2)	presente → pres = 2; aus = 0	—



6	marca = A (e=1, d=3)	ausente → pres = 2; aus = 1	—
7	marca = P (e=1, d=4)	presente → pres = 3; aus = 1	—
8	—	d = 5 > 4: fin del ciclo interno; totP = 3; totA = 1	Estudiante 1 -> Asistencias: 3 Ausencias: 1
9	—	e = 2: pres = 0; aus = 0; d = 1	—
10	marca = A (e=2, d=1)	ausente → pres = 0; aus = 1	—
11	marca = A (e=2, d=2)	ausente → pres = 0; aus = 2	—
12	marca = x (e=2, d=3)	no es P ni A → inválida; se vuelve a solicitar	Error: ingrese solo P o A.
13	marca = P (e=2, d=3)	presente → pres = 1; aus = 2	—
14	marca = P (e=2, d=4)	presente → pres = 2; aus = 2	—
15	—	d = 5 > 4: fin del ciclo interno; totP = 5; totA = 3	Estudiante 2 -> Asistencias: 2 Ausencias: 2
16	—	e = 3: pres = 0; aus = 0; d = 1	—
17	marca = P (e=3, d=1)	presente → pres = 1; aus = 0	—
18	marca = P (e=3, d=2)	presente → pres = 2; aus = 0	—
19	marca = P (e=3, d=3)	presente → pres = 3; aus = 0	—
20	marca = P (e=3, d=4)	presente → pres = 4; aus = 0	—
21	—	d = 5 > 4: fin del ciclo interno; totP = 9; totA = 3	Estudiante 3 -> Asistencias: 4 Ausencias: 0
22	—	e = 4 > 3: termina el ciclo externo	TOTAL DEL CURSO -> Asistencias: 9 Ausencias: 3

2.7.10 Ejercicio 10: Sistema integrado de ventas

Problema

Desarrollar un sistema con menú: 1) Registrar venta, 2) Mostrar estadísticas y 3) Salir. Cada venta debe solicitar producto, cantidad y precio. Las estadísticas mostrarán número de ventas, unidades vendidas, total recaudado, venta mayor y promedio por venta.

Análisis

Estructura repetitiva utilizada: do-while + for/while.

Se integran los ciclos: do-while mantiene el menú activo hasta elegir Salir, while valida cantidad y precio, y for recorre los arreglos para listar las ventas.

Tabla 19. Entradas, procesos y salidas del Ejercicio 10.

Entradas	Procesos	Salidas
• opción del menú (entero) • producto (texto de una palabra), cantidad (entero > 0), precio (real > 0)	• monto = cantidad × precio • nVentas++; unidades += cantidad; total += monto • Actualizar la venta mayor • Promedio = total / nVentas • for para listar las ventas registradas	• Confirmación de cada ventaregistrada • Detalle de ventas • Número de ventas, unidadesvendidas, total recaudado, venta mayor y promedio por venta

Algoritmo

1. Inicio.



2. Establecer $nVentas = 0$, $unidades = 0$, $total = 0$ y $ventaMayor = 0$.

3. Repetir:

Mostrar el menú y leer opción.

Opción 1: leer producto, cantidad y precio (validando > 0); $monto = cantidad \times precio$; guardar producto y monto; actualizar $nVentas$, $unidades$, $total$ y $ventaMayor$.

Opción 2: si $nVentas = 0$ informar que no hay ventas; si no, listar las ventas con un for y mostrar $nVentas$, $unidades$, $total$, $ventaMayor$ y $total / nVentas$.

Otra opción distinta de 3: «Opción inválida».

4. Mientras opción $\neq 3$.

5. Mostrar «Fin del programa».

6. Fin.

Pseudocódigo

```
Algoritmo Ejercicio10
    Definir opcion, cantidad, n, unidades, i Como Entero
    Definir precio, monto, total, ventaMayor Como Real
    Definir producto Como Caracter
    Dimension productos[100], montos[100]
    n <- 0; unidades <- 0; total <- 0; ventaMayor <- 0
    Repetir
        Escribir "1) Registrar venta 2) Mostrar estadísticas 3) Salir"
        Leer opcion
        Si opcion = 1 Entonces
            Escribir "Producto: "
            Leer producto
            Escribir "Cantidad: "
            Leer cantidad
            Mientras cantidad <= 0 Hacer
                Escribir "Cantidad inválida. Ingrese un valor mayor que 0: "
                Leer cantidad
            FinMientras
            Escribir "Precio unitario: $"
            Leer precio
            Mientras precio <= 0 Hacer
                Escribir "Precio inválido. Ingrese un valor mayor que 0: $"
                Leer precio
            FinMientras
            monto <- cantidad * precio          n <- n + 1
            productos[n] <- producto            montos[n] <- monto
            unidades <- unidades + cantidad    total <- total + monto
            Si monto > ventaMayor Entonces      ventaMayor <- monto
            FinSi
            Escribir "Venta registrada: $", monto
        SiNo
            Si opcion = 2 Entonces
                Si n = 0 Entonces
                    Escribir "Aún no hay ventas registradas."
                SiNo
                    Para i <- 1 Hasta n Con Paso 1 Hacer
                        Escribir i, " ", productos[i], " - $", montos[i]
                    FinPara
                    Escribir "Número de ventas: ", n
                    Escribir "Unidades vendidas: ", unidades
                    Escribir "Total recaudado: $", total
                    Escribir "Venta mayor: $", ventaMayor
                    Escribir "Promedio por venta: $", total / n
                FinSi
            SiNo
                Si opcion <> 3 Entonces
                    Escribir "Opción inválida."
                FinSi
            FinSi
        FinSi
    Hasta Que opcion = 3
    Escribir "Fin del programa."
FinAlgoritmo
```

Diagrama de flujo

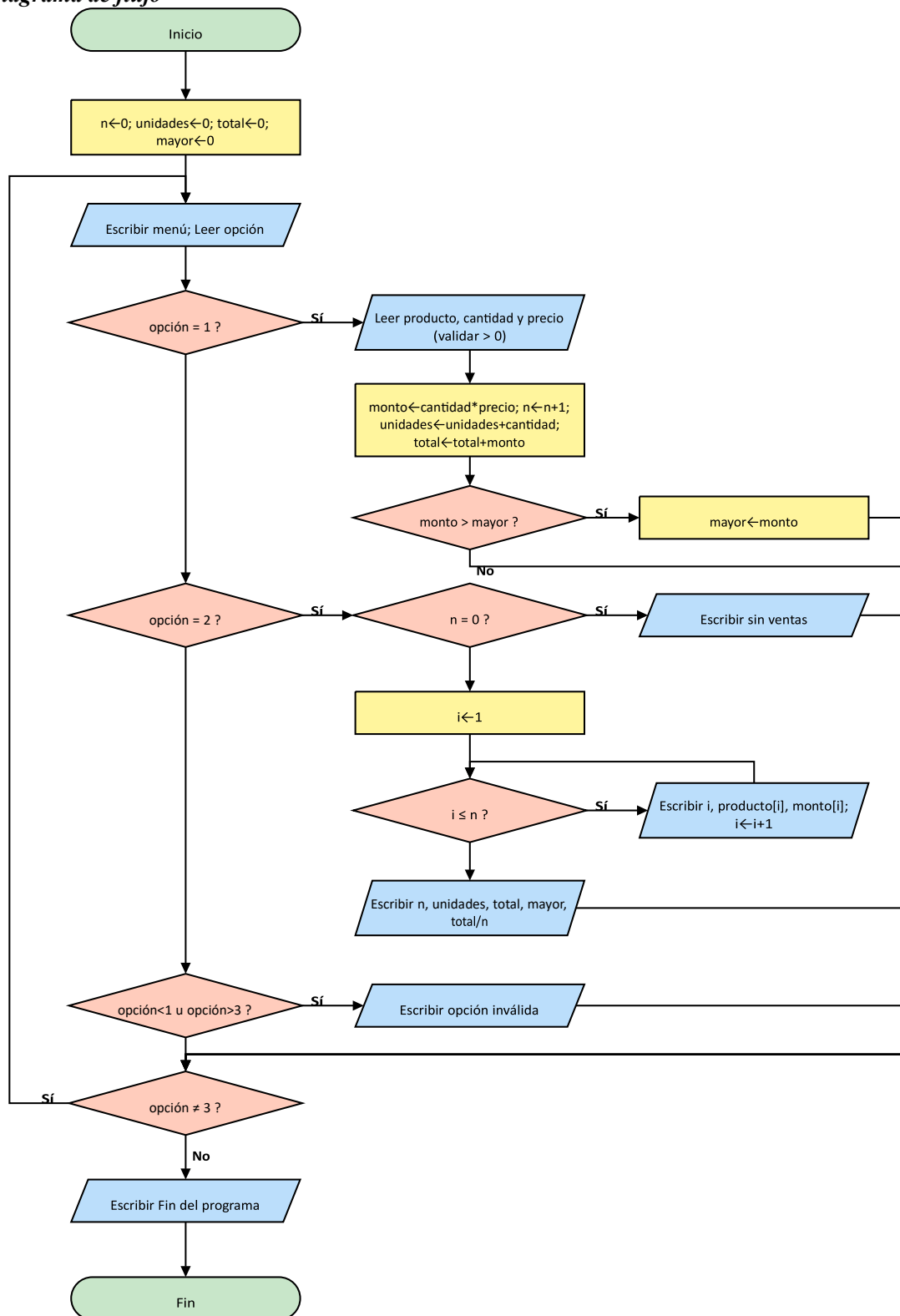


Figura 10. Diagrama de flujo del Ejercicio 10: Sistema integrado de ventas.

Código Java

Archivo Ejercicio10.java



```
import java.util.Locale;
import java.util.Scanner;

public class Ejercicio10 {
    public static void main(String[] args) {
        Scanner sc = new
Scanner(System.in).useLocale(Locale.US);

        final int MAX = 100;
        String[] productos = new String[MAX];
        double[] montos = new double[MAX];

        int numeroVentas = 0, unidades = 0;
        double total = 0, ventaMayor = 0;
        int opcion;

        do {
            System.out.println("\n--- SISTEMA DE VENTAS -
--");
            System.out.println("1) Registrar venta 2) Mostrar
estadisticas 3) Salir");
            System.out.print("Opcion: ");
            opcion = sc.nextInt();

            if (opcion == 1) {
                if (numeroVentas == MAX) {
                    System.out.println("Capacidad maxima de
ventas alcanzada.");
                } else {
                    System.out.print("Producto: ");
                    String producto = sc.next();

                    System.out.print("Cantidad: ");
                    int cantidad = sc.nextInt();

                    while (cantidad <= 0) {
                        System.out.print("Cantidad invalida.
Ingrese un valor mayor que 0: ");
                        cantidad = sc.nextInt();
                    }

                    System.out.print("Precio unitario: $");
                    double precio = sc.nextDouble();

                    while (precio <= 0) {
                        System.out.print("Precio invalido. Ingrese
un valor mayor que 0: $");
                        precio = sc.nextDouble();
                    }

                    double monto = cantidad * precio;
```



```
        productos[numeroVentas] = producto;
        montos[numeroVentas] = monto;
        numeroVentas++;
        unidades += cantidad;
        total += monto;

        if (monto > ventaMayor) ventaMayor =
monto;

        System.out.printf("Venta registrada:
        $%.2f%n", monto);
    }

    } else if (opcion == 2) {
        if (numeroVentas == 0) {
            System.out.println("Aun no hay ventas
registradas.");
        } else {
            System.out.println("Detalle de ventas:");

            for (int i = 0; i < numeroVentas; i++) {
                System.out.printf(" %d. %s - $%.2f%n",
                    i + 1, productos[i], montos[i]);
            }

            System.out.println("Numero de ventas: " +
numeroVentas);
            System.out.println("Unidades vendidas: " +
unidades);
            System.out.printf("Total recaudado:
            $%.2f%n", total);
            System.out.printf("Venta mayor: $%.2f%n",
ventaMayor);
            System.out.printf("Promedio por venta:
            $%.2f%n",
                total / numeroVentas);
        }

    } else if (opcion != 3) {
        System.out.println("Opcion invalida.");
    }

    } while (opcion != 3);

    System.out.println("Fin del programa.");
}
}
```

Prueba de escritorio

Datos de prueba: opciones 2 (sin ventas), 1 (Cuaderno, 3, 2.00), 1 (Mochila, 1, 25.00), 1 (Lapiz, 10, 0.50), 2 y 3.

Tabla 20. Prueba de escritorio del Ejercicio 10.

N.º	Entrada	Proceso	Salida
-----	---------	---------	--------



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL CARRERA
DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



1	—	nVentas = 0; unidades = 0; total = 0; ventaMayor = 0	—
2	opción = 2	nVentas = 0 → no hay ventas para mostrar	Aun no hay ventas registradas.
3	opción = 1; Cuaderno; 3; 2.00	monto = $3 \times 2.00 = 6.00$; nVentas = 1; unidades = 3; total = 6.00; $6.00 > 0 \rightarrow$ ventaMayor = 6.00	Venta registrada: \$6.00
4	opción = 1; Mochila; 1; 25.00	monto = 25.00; nVentas = 2; unidades = 4; total = 31.00; ventaMayor = 25.00	Venta registrada: \$25.00
5	opción = 1; Lapiz; 10; 0.50	monto = 5.00; nVentas = 3; unidades = 14; total = 36.00; $5.00 < 25.00 \rightarrow$ ventaMayor no cambia	Venta registrada: \$5.00
6	opción = 2	nVentas = 3 → el for recorre las 3 ventas; promedio = $36.00 / 3 = 12.00$	1. Cuaderno - \$6.00; 2. Mochila - \$25.00; 3. Lapiz - \$5.00; ventas 3; unidades 14; total \$36.00; mayor \$25.00; promedio \$12.00
7	opción = 3	opción = 3 → condición falsa; termina el do-while	Fin del programa.



2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☒ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- Se resolvieron los diez ejercicios de la guía APE 3 siguiendo la secuencia problema, análisis, algoritmo, pseudocódigo, diagrama de flujo, código Java y prueba de escritorio, lo que permitió verificar la lógica de cada solución antes y después de programarla.
- La estructura repetitiva se eligió según la naturaleza de cada problema: for cuando el número de repeticiones se conoce de antemano (ejercicios 1, 6 y 9); while cuando la repetición depende de un centinela o de una validación previa (ejercicios 2, 4 y 8); y do-while cuando el bloque debe ejecutarse al menos una vez, como en los menús y en el registro de ventas (ejercicios 3, 5, 7 y 10).
- El uso de contadores, acumuladores, centinelas y validaciones de rango evitó datos incorrectos (notas fuera de 0 a 10, edades no válidas, división entre cero, retiros mayores al saldo) sin interrumpir el programa.
- Las pruebas de escritorio, elaboradas con datos que incluyen casos inválidos y de frontera, coincidieron con la ejecución real de cada programa, por lo que sirven como evidencia del correcto funcionamiento.

2.10 Recomendaciones

- Probar cada programa con valores límite (por ejemplo 0, 7, 10, 12 y -1) además de los datos típicos, para comprobar que las condiciones de los ciclos y de las validaciones son las correctas.
- Definir con constantes los valores fijos (nota de aprobación, tarifas del estacionamiento, saldo inicial) para facilitar su modificación.
- Los programas asumen que el usuario escribe números cuando se espera un número; como mejora, se puede comprobar la entrada con hasNextInt() o hasNextDouble() antes de leerla.
- Versionar cada ejercicio en el repositorio de GitHub con mensajes de commit descriptivos, según lo indicado en la guía.

2.11 Anexos

Anexo A. Resumen de estructuras repetitivas por ejercicio.

Tabla 21. Estructuras y conceptos aplicados en cada ejercicio.

N.º	Ejercicio	Estructura principal	Estructura auxiliar	Conceptos
1	Promedio de calificaciones	for	do-while (validación)	Contador, acumulador, mayor y menor
2	Control de edades	while	—	Centinela (-1), contadores, acumulador
3	Calculadora con menú	do-while	switch	Menú, control de división entre cero



4	Tabla de multiplicar	while	for	Validación previa, ciclo controlado
5	Cajero universitario	do-while	switch	Menú, acumulador (saldo), contador
6	Estadísticas de un curso	for	while (validación)	Acumuladores, porcentajes
7	Venta de entradas	do-while	while (validación)	Acumulador, contador, repetición S/N
8	Estacionamiento	while	—	Centinela (0), acumulador, selección
9	Matriz de asistencia	for anidado	do-while (validación)	Ciclos anidados, contadores
10	Sistema integrado de ventas	do-while	for y while	Menú, acumuladores, arreglos

Anexo B. Organización del repositorio GitHub (guía APE 3).

